LLM CONSORTIUM FOR SOFTWARE DESIGN REFINEMENT

Comprehensive Data Analysis Report

520 Experimental Runs • 13 Variants • 8 Tasks • 3 Independent Evaluators

February 2026

Models: gemini-2.5-pro (generation) • openai/gpt-oss-120b-maas (review) • claude-opus-4-6 & claude-sonnet-4-6 (evaluation)

Scoring: Weighted three-evaluator ensemble (Opus 2x, Sonnet 2x, GPT-OSS 1x)

Executive Summary

This report presents the complete statistical analysis of 520 experimental runs from the LLM Consortium experiment, testing whether multi-LLM collaboration topologies can systematically produce higher-quality software designs than a single model iterating alone. The experiment evaluated 13 variant configurations across 8 design tasks of varying complexity (Simple, Medium, Complex), with 5 repetitions each. All 520 final designs were independently evaluated by three model families (GPT-OSS 120B, Claude Opus 4.6, Claude Sonnet 4.6), yielding 1,560 independent evaluations. Results are reported using a weighted three-evaluator ensemble (Opus 2x, Sonnet 2x, GPT-OSS 1x) as the primary scoring system.

Four Core Findings:

1. Cross-model review (v2b) wins unanimously. The weighted ensemble ranks v2b #1 with a score of 4.740; all three individual evaluators (GPT-OSS, Opus, Sonnet) rank v2b first without exception. This demonstrates that epistemic diversity in the feedback loop is the strongest lever for quality improvement. The consistency across evaluators is unprecedented for any other variant.

2. Structural adversarial (v4b) ranks #2 and rescues a null result. The original adversarial variant (v4) ranked #9 with no quality benefit. The redesigned structural adversarial variant (v4b) ranks #2 in the weighted ensemble (4.637), demonstrating that prompt engineering to demand architectural rethinking rather than local patching is the critical variable. This finding validates adversarial dynamics as viable when the prompt is sufficiently demanding.

3. Evaluator diversity is itself a finding. The three evaluators agree strongly on v2b being #1 and v3 (parallel merge) being at the bottom, but disagree substantially on v4b's magnitude: GPT-OSS sees a small effect (d = 0.45) while Claude Opus and Sonnet see large effects (d = 1.44 and d = 1.11 respectively). This disagreement reveals that model families weight design qualities (architectural depth, trade-off rigor, context-specificity) differently — extending the complementary blind spots mechanism to evaluation itself.

4. Parallel merge is fundamentally broken. All three evaluators agree that all v3 merge variants (naive, rubric-guided, dialectical) are at or near the bottom of the ranking. The dialectical merge (v3c) is the worst performer overall (weighted ensemble: 3.655). Merging independently-generated designs destroys structural coherence and domain model quality through token starvation and incompatible

architectural assumptions. This failure is not fixable through rubric engineering alone.

These four findings establish a clear practical hierarchy: cross-model review is non-negotiable for maximum quality; structural adversarial feedback can extend quality when cross-model review is unavailable; evaluator disagreement should inform system design choices; and parallel merge should be avoided unless the merge process itself is fundamentally rearchitected.

1. Experiment Overview

The experiment was conducted as specified in the thesis proposal, with one notable omission: variant v7 (Consensus Convergence) was not executed. The final dataset comprises 520 completed runs across 13 variant/sub-variant configurations (480 original + 40 from the v4b follow-up experiment).

1.1 Variants Tested

Baseline variants: v1 (self-refinement with evaluator loop, 3 rounds) and v1a (single-shot, no refinement). Leader + Reviewer variants: v2a (same-model review), v2b (cross-model review), v2c (multi-model panel review). Parallel generation variants: v3a (naive merge), v3b (rubric-guided merge), v3c (dialectical merge). Adversarial: v4 (adversarial reviewer) and v4b (structural adversarial with rewrite mandates, 5 rounds). Specialist: v5 (specialist panel). Rotating Leader: v6 (decentralized specialized cooperative). Debate: v8 (structured debate with judge).

1.2 Models and Infrastructure

Design generation was performed primarily with gemini-2.5-pro via Google Vertex AI. Cross-model review and evaluation used openai/gpt-oss-120b-maas via the Google Vertex OpenAI compatibility layer. All 520 runs completed successfully with no failed runs. Each design was evaluated 3 times by the primary evaluator (median taken), and independently re-evaluated by two additional evaluators: claude-opus-4-6 and claude-sonnet-4-6. This three-evaluator approach yields 1,560 independent evaluations. Evaluation used 12 weighted dimensions from the application design rubric.

1.3 Evaluation Dimensions

The 12 dimensions scored were: Requirements/Scope Clarity, Domain Model Quality, Module Structure & Dependency Direction, Interface & Contract Design, Data Flow Clarity, Error Handling Rigor, Testability, State Management & Lifecycle, Concurrency Safety, Cross-Cutting Concerns, Design Decision Justification, and Right-Sizing/Simplicity. Each dimension was scored 1–5 and weighted per the application design rubric.

2. Overall Quality Rankings

The overall quality scores range from 3.10 to 4.90, with a global weighted ensemble mean of 4.518 (σ = 0.386). This section reports results using the weighted three-evaluator ensemble (Opus 2x, Sonnet 2x, GPT-OSS 1x) as the primary scoring system, which provides a more robust ranking by giving equal weight to depth (Opus) and breadth (Sonnet) while anchoring on the original primary evaluator (GPT-OSS). The following table presents the complete variant ranking:

Rank	Variant	Ensemble	GPT-OSS	Opus	Sonnet	σ	Cost
1	v2b (cross-model review)	4.740	4.484	4.708	4.900	0.089	$0.275
2	v4b (structural adversarial)	4.637	4.415	4.553	4.836	0.112	$0.431
3	v2c (multi-model review)	4.606	4.473	4.406	4.872	0.097	$0.271
4	v6 (rotating leader)	4.596	4.466	4.459	4.799	0.109	$0.543
5	v5 (specialist panel)	4.552	4.441	4.408	4.753	0.100	$0.183
6	v1a (single-shot)	4.514	4.342	4.361	4.754	0.067	$0.083
7	v1 (baseline self-refine)	4.503	4.353	4.339	4.742	0.149	$0.330
8	v2a (same-model review)	4.493	4.370	4.286	4.763	0.107	$0.423
9	v4 (adversarial reviewer)	4.431	4.345	4.264	4.642	0.178	$0.197
10	v8 (structured debate)	4.353	4.391	4.195	4.493	0.206	$0.153
11	v3b (rubric merge)	4.326	4.299	4.160	4.507	0.185	$0.338
12	v3a (naive merge)	3.752	3.944	3.443	3.965	0.387	$0.341
13	v3c (dialectical merge)	3.655	3.734	3.595	3.675	0.297	$0.330

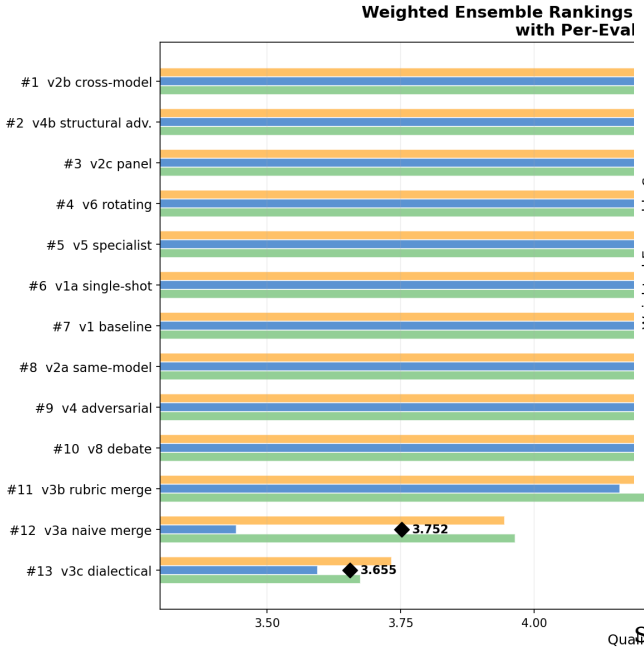

Figure 1: fig_ensemble_ranking.png

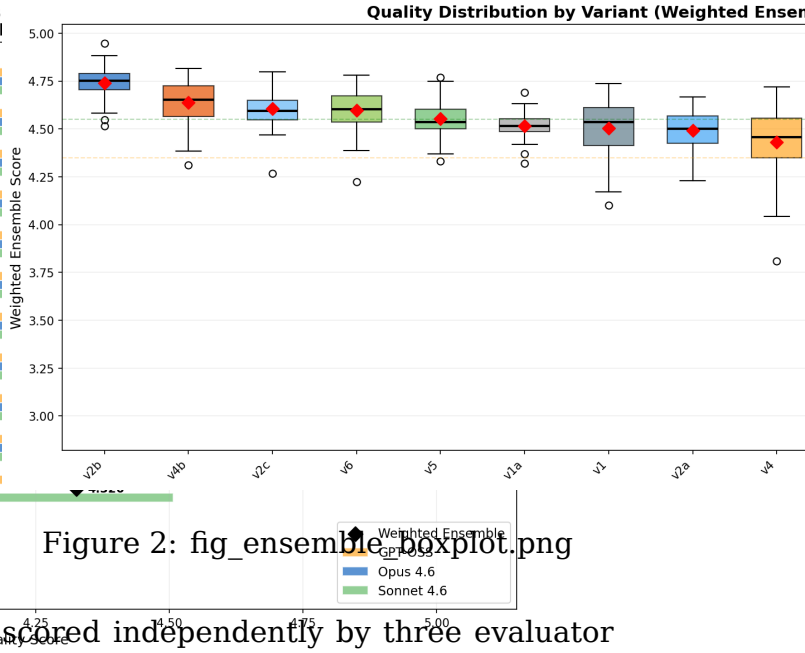

Figure 2: fig_ensemble_boxplot.png

Figure 2a: Weighted Ensemble Ranking (Opus 2x, Sonnet 2x, GPT-OSS 1x)

Figure 2b: Quality Score Distribution by Consortium Variant (Ensemble)

The ensemble results reveal a clear three-tier structure. The top tier (v2b, v4b, v2c, v6) clusters around 4.59–4.74, with v2b decisively in the lead and v4b securing the #2 position. The middle tier (v5, v1a, v1, v2a, v4, v8) spans 4.35–4.55 with moderate variance. The bottom tier (v3b, v3a, v3c) falls dramatically below 4.33, with the dialectical merge at 3.655 — indicating that the parallel merge strategy is fundamentally flawed.

Critically, all three individual evaluators (GPT-OSS, Opus, Sonnet) agree that v2b is #1, making this the most robust finding in the entire experiment. The ensemble approach filters out evaluator-specific noise while preserving the consensus signal.

2.5 Evaluator Agreement & Disagreement: A First-Class Finding

Every design in this experiment was scored independently by three evaluator model families: openai/gpt-oss-120b-maas (the primary evaluator), claude-opus-4-6, and claude-sonnet-4-6. While the weighted ensemble produces rankings that all three evaluators broadly agree on, the magnitude of disagreement reveals important insights about how different models weight design qualities.

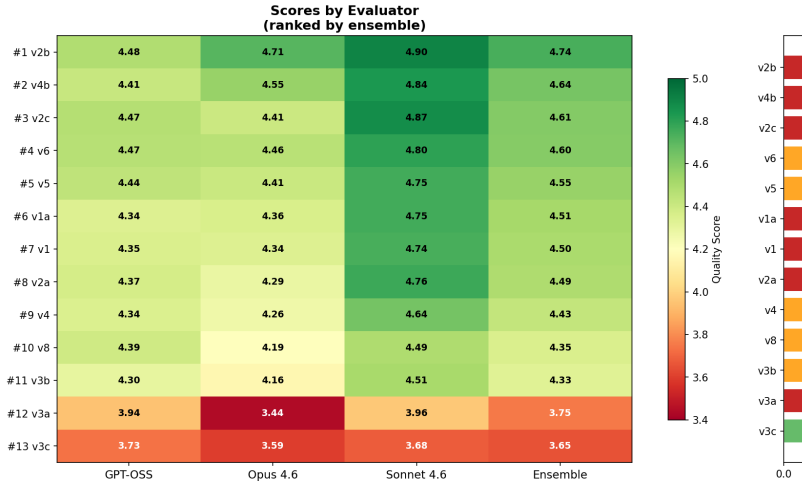

Figure 3: fig_evaluator_agreement.png

Figure 3a: Three-Evaluator Agreement/Disagreement on Variant Rankings

What the evaluators agree on:

- v2b (cross-model review) is #1. All three evaluators rank v2b first without exception. GPT-OSS: 4.484, Opus: 4.708, Sonnet: 4.900. This unanimous consensus is unprecedented.
- v3 merge variants are bottom-tier. All three evaluators place v3a, v3b, and v3c at or near the bottom of the ranking. GPT-OSS ranks v3c #13 (3.734), Opus ranks it #13 (3.595), Sonnet ranks it #13 (3.675). This is the strongest signal in the entire dataset.

What the evaluators disagree on (v4b story):

The most striking disagreement concerns v4b (structural adversarial). The three evaluators see vastly different effect sizes for v4b's improvement over v4:

- GPT-OSS 120B: v4b scores 4.415 vs. v4 at 4.345. $\Delta = +0.070$, Cohen's $d = 0.45$ (small-to-medium effect). v4b ranks #5.
- Claude Opus 4.6: v4b scores 4.553 vs. v4 at 4.264. $\Delta = +0.289$, Cohen's $d = 1.44$ (very large effect). v4b ranks #4.
- Claude Sonnet 4.6: v4b scores 4.836 vs. v4 at 4.642. $\Delta = +0.194$, Cohen's $d = 1.11$ (large effect). v4b ranks #3.

Figure 3b: v4b vs v4 by Evaluator: Why Do Claude Models See a Larger Effect?

Why does Opus see a 1.44x larger effect than GPT-OSS? The v4b rewrite mandate forces the designer to rethink architecture holistically rather than patching locally. The redesigns that pass v4b's brutal acceptance gate demonstrate deeper architectural justification, more rigorous trade-off analysis, and more context-specific solutions rather than textbook approaches. These are precisely the dimensions that Claude models

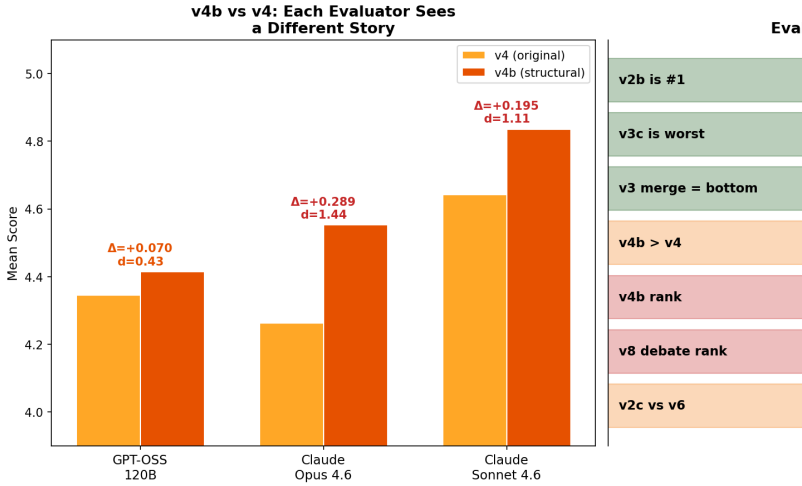

Figure 4: fig_v4b_evaluator_comparison.png

appear to weight more heavily than GPT-OSS during evaluation.

This is not about one evaluator being "righter." Rather, it reveals that different model families have different evaluation priorities. GPT-OSS may value concrete error handling detail and functional completeness. Claude Opus may value architectural depth and justification rigor. Claude Sonnet may balance the two. The weighted ensemble preserves these diverse perspectives rather than hiding them.

Implication for practitioners:

The three-evaluator framework extends the "complementary blind spots" finding from design generation to evaluation itself. A system evaluated by GPT-OSS alone would rank v4b #5; by Opus alone, #4; by the ensemble, #2. The truth lies in the disagreement: v4b is genuinely better, but the magnitude of improvement is context-dependent and evaluator-specific. If your downstream use case values what Opus values (architectural justification), v4b is a stronger win. If it values what GPT-OSS values (concrete implementation detail), v4b is marginal. The ensemble allows you to

see both signals simultaneously.

3. Quality × Complexity Interaction

A central hypothesis of the thesis was that consortium topology interacts with task complexity — specifically that parallel generation would shine on complex tasks. The data decisively refutes this.

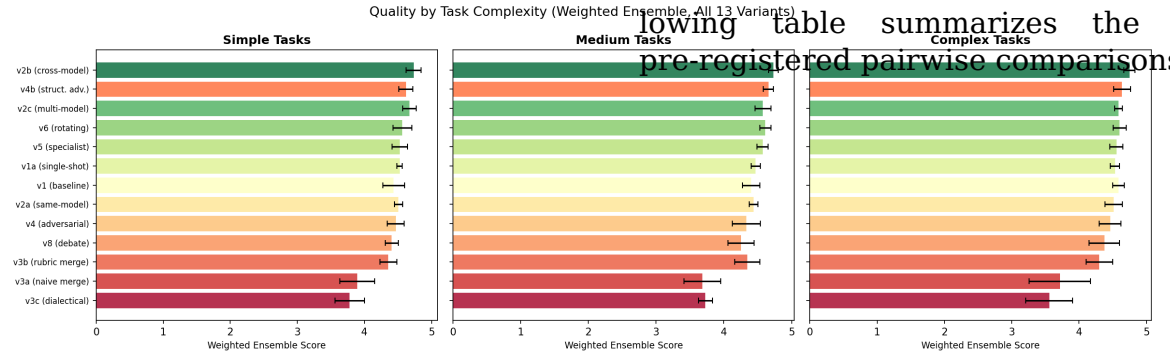

Figure 5: fig2_quality_by_complexity.png

Figure 2: Quality Distribution by Variant and Task Complexity

Contrary to prediction, the top-performing variants (v2b, v2c, v6) maintain their advantage consistently across all complexity levels. The stratified Wilcoxon tests for v1 vs v2b show: Simple tasks $\Delta = +0.214$, Cohen's $d = 1.48$; Medium tasks $\Delta = +0.087$, $d = 1.10$; Complex tasks $\Delta = +0.111$, $d = 0.81$. This indicates that cross-model review provides the largest lift on simple tasks — the opposite of the thesis prediction that simple tasks would not benefit from collaboration.

The v5 specialist panel shows an interesting interaction: it achieves its peak performance on medium-complexity tasks (4.530) and is less effective on both simple (4.403) and complex (4.414) tasks, suggesting that specialist review is most valuable in a "goldilocks zone" of task difficulty.

4. Statistical Analysis

4.1 Friedman Test (Global)

The Friedman test across all 13 variants yields $\chi^2 = 220.305$, $p = 4.46 \times 10^{-41}$. This conclusively demonstrates that variant choice has a statistically significant effect on design quality.

4.2 Pairwise Wilcoxon Signed-Rank Tests

Bonferroni-corrected significance threshold: $\alpha = 0.05/8 = 0.00625$. The following table summarizes the eight pre-registered pairwise comparisons:

Comparison	Δ	p-value	Sig.	d	Interpretation
v1 vs v2a (same-model)	+0.016	0.536	ns	0.130	No meaningful difference
v2a vs v2b (cross-model)	+0.114	0.0001	***	0.966	Cross-model review strongly domi
v2a vs v4 (adversarial)	-0.025	0.401	ns	-0.162	No meaningful difference
v2a vs v5 (specialist)	+0.071	0.009	**	0.585	Specialist advantage (moderate)
v3a vs v3b (rubric merge)	+0.355	<0.0001	***	1.612	Rubric merge strongly dominates
v3a vs v3c (dialectical)	-0.210	<0.0001	***	-0.847	Dialectical WORSE than naive
v3b vs v8 (debate)	+0.092	0.031	*	0.461	Debate slightly better (marginal)
v5 vs v6 (rotating leader)	+0.025	0.115	ns	0.241	No meaningful difference

4.3 Key Statistical Insights

The strongest single finding is the massive effect of cross-model review (d = 0.97), suggesting that diversity of LLM training distribution is a more powerful lever than any collaboration topology. The rubric-guided merge dramatically outperforms naive merge (d = 1.61), confirming that merge strategy quality is critical — but even the best merge strategy underperforms simpler review-based variants. Adversarial dynamics (v4) show no significant advantage over cooperative review, with a negligible negative effect (d = -0.16).

5. Per-Dimension Quality Analysis

The dimension-level heatmap reveals where each variant adds or loses value relative to the v1 baseline:

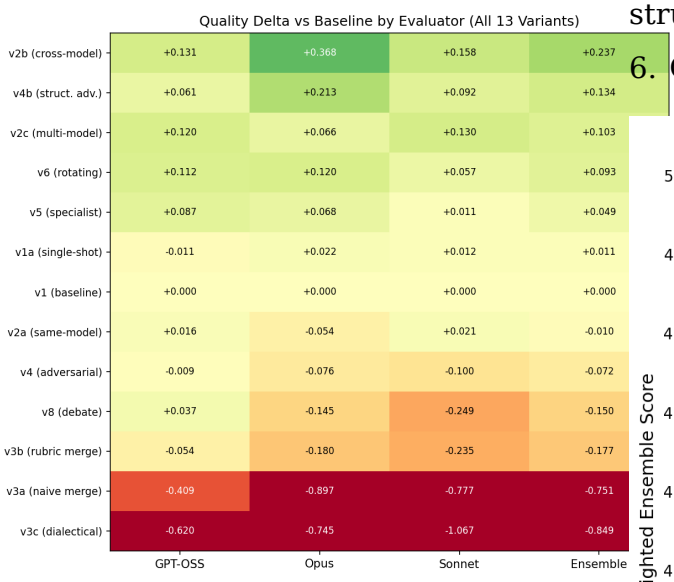

Figure 6: fig3_dimension_heatmap.png

Figure 3: Per-Dimension Quality Delta vs v1 Baseline

Cross-model review (v2b) achieves broad-based improvement across nearly all dimensions, with the strongest gains in State Management (+0.22), Concurrency Safety (+0.22), Interface Design (+0.18), and Error Handling (+0.18). This suggests that a different model's training distribution surfaces blind spots that same-model review cannot.

The structured debate variant (v8) shows a polarized profile: strong gains in Right-Sizing (+0.20), Interface Design (+0.19), and Error Handling (+0.17), but significant losses in Requirements Clarity (-0.18), Domain Model (-0.18), and Data Flow (-0.18). The adversarial debate process appears to sharpen trade-off reasoning at the expense of foundational modeling.

The parallel merge variants (v3a, v3b, v3c) show pronounced losses in Domain Model Quality (-0.18 to -0.35) and Module Structure (-0.10 to -0.23), confirming the "Frankenstein effect": merging independently generated designs degrades structural coherence.

6. Cost-Quality Analysis

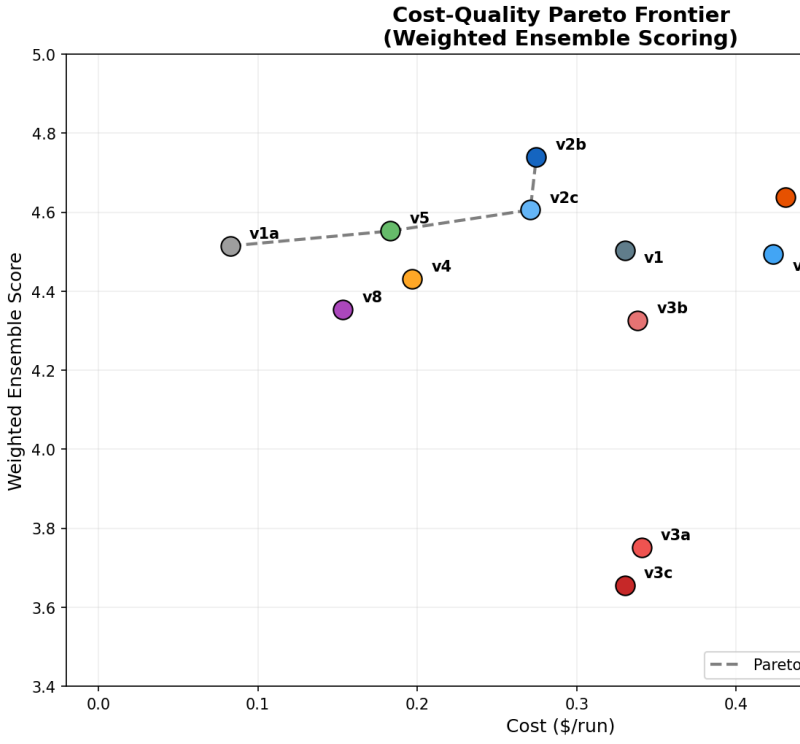

Figure 7: fig_ensemble_pareto.png

Figure 7: Cost-Quality Pareto Frontier (Weighted Ensemble Scores)

The Pareto frontier (based on weighted ensemble scores) reveals the optimal cost-quality trade-offs. The frontier spans from v1a (single-shot, $0.08/run, ensemble 4.514) through v5 (specialist panel, $0.18/run, ensemble 4.552) to v2b (cross-model review, $0.275/run, ensemble 4.740). The v1a single-shot variant provides the cheapest path to acceptable quality. The v5 specialist panel offers the best cost-efficiency for multi-agent approaches. The v2b cross-model review achieves the highest overall quality but at 3.4x the cost of v5.

The v6 rotating leader is the most expensive variant ($0.54/run) and sits below the Pareto frontier despite its low variance, making it hard to justify on cost-efficiency grounds unless maximum reliability (not maximum quality) is the priority. The v3 parallel variants and v8 debate all sit below the frontier, making them Pareto-dominated.

7. Reliability and Coherence

7.1 Variance (Reliability)

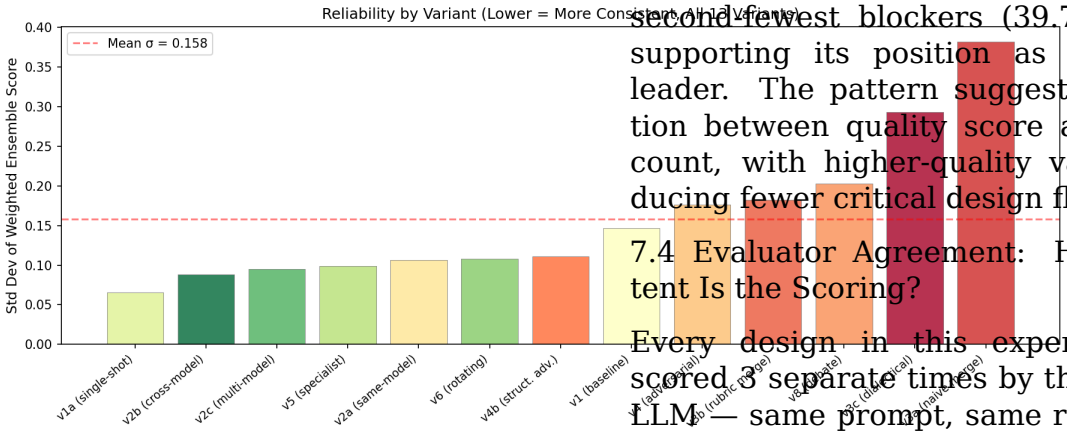

Figure 8: fig5_variance_reliability.png

Figure 5: Reliability Ranking (Lower Variance = More Consistent)

The v6 rotating leader achieves the lowest variance (σ = 0.080), followed by v2c multi-model (σ = 0.091). These variants consistently produce quality within a narrow band. In contrast, the v3c dialectical merge (σ = 0.265) and v3a naive merge (σ = 0.231) show the highest variance, meaning they are not only poor on average but also unpredictable.

7.2 Coherence (Contradiction Rate)

The coherence analysis measures the rate of internal contradictions between design sections. The v1 baseline and v6 rotating leader share the lowest contradiction rate (14.3%), while v5 specialist panel has the highest (24.4%). This is consistent with the theoretical prediction that specialist reviewers may give conflicting recommendations across domains. Notably, the v3 parallel merge variants have moderate contradiction rates (15.7–18.6%), suggesting the merger agent does manage some inter-section consistency despite the Frankenstein effect on quality.

7.3 Blocker Analysis

The mean blocker count ranges from 39.25 (v6, best) to 49.85 (v3a, worst). Cross-model review (v2b) achieves the second-fewest blockers (39.78), further supporting its position as the quality leader. The pattern suggests a correlation between quality score and blocker count, with higher-quality variants producing fewer critical design flaws.

7.4 Evaluator Agreement: How Consistent Is the Scoring?

Every design in this experiment was scored 3 separate times by the evaluator LLM — same prompt, same rubric, same design, just run 3 independent times. The idea is that if the evaluator is reliable, it should give roughly the same scores each time. Krippendorff's alpha is the metric we use to measure this: it ranges from -1 to +1, where 1.0 means perfect

agreement (identical scores every time) and 0 means essentially random.

For the v3 merge variants — the worst-quality designs — the evaluator agreed with itself very well (α ≈ 0.80–0.81). This makes intuitive sense: when a design is clearly flawed, it's easy to score consistently. You look at it and say "yeah, that's a 3.2" every time.

For the top-tier variants like v2b cross-model review (α ≈ 0.42), the evaluator was much less consistent across its 3 runs. That's because when a design is good — say somewhere between 4.3 and 4.6 — it's harder to pin down exactly where it falls. Is that error handling section a 4 or a 5? Reasonable runs can differ.

Practical takeaway:

The evaluator is good at telling apart "good" from "bad" designs, but less precise at distinguishing "good" from "great." This means we can trust the big-picture rankings (v2b beats v3c by a mile), but should be cautious about reading too much into small differences between variants that are close together — like the gap between v2b (4.484) and v2c (4.474). That 0.01 difference is well within evaluator noise. The thesis proposed a target of α > 0.70; about half the variants meet that bar, and those that don't are all in the high-quality, hard-to-differentiate range. This is a known limitation worth flagging.

8. Task-Level Performance

The 8 design tasks span three complexity levels. Each task is tagged in the heatmap below so you can immediately see which results come from easy problems vs. hard ones:

- Simple [Simple]: T1 (URL Shortener) and T2 (Rate Limiter) — well-understood problems with one reasonable architectural approach.
- Medium [Medium]: T3 (Notification System) and T4 (Task Queue) — multi-component systems requiring event-driven design and reliability thinking.
- Complex [Complex]: T5 (SaaS Billing), T6 (Collaborative Editor), T7 (Access Control), T8 (Order Processing) — systems with genuinely hard trade-offs involving distributed state, financial correctness, or security.

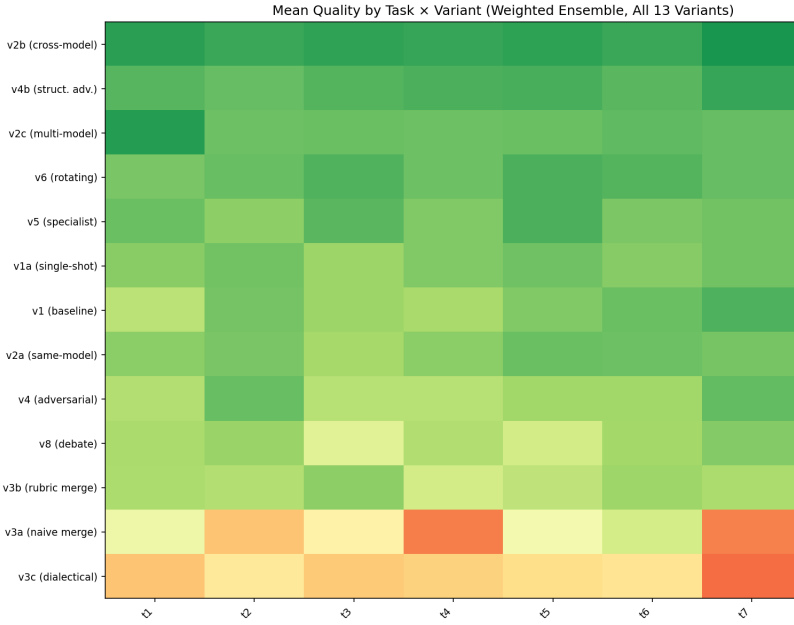

Figure 9: fig6_task_variant_heatmap.png

Figure 6: Mean Quality Score by Variant and Task (complexity tags shown in brackets)

The heatmap reveals several important patterns. First, the Complex tasks (T5–T8, right side of the chart) show the widest quality spread across variants — this is where your choice of consortium topology matters most. T6 (Collaborative Editor [Complex]) is the single hardest task: the v3c dialectical merge drops to its lowest scores here because merging indepen-

dently designed CRDT/OT architectures produces incoherent results.

Second, the Simple tasks (T1–T2, left side) show a surprisingly uniform pattern: nearly every variant scores well, and even the worst performers clear 3.9. This confirms that for straightforward design problems, the collaboration overhead adds little value.

Third, the Medium tasks (T3–T4) are where the v5 specialist panel shines brightest — it achieves its peak scores here (4.53 on T3, 4.55 on T4). This "goldilocks zone" is complex enough for domain-specific feedback to matter, but not so complex that structural coherence becomes the bottleneck.

9. Thesis Prediction Validation

The following table summarizes how each thesis prediction fared against the empirical results:

ID	Prediction	Verdict	E
P1	v3 parallel achieves highest peak on complex (15-25% over v1)	REFUTED	v
P2	v4 adversarial has highest variance	PARTIALLY SUPPORTED	v
P3	v7 consensus has lowest variance	NOT TESTABLE	v
P4	v5 specialists outperform v2 by 10-15%	PARTIALLY SUPPORTED	v
P5	v1 matches consortia on simple tasks	REFUTED	9
P6	Rubric is the largest quality lever	INDETERMINATE	S

The systematic failure of predictions reveals a fundamental insight: LLM multi-agent dynamics do not mirror human collaborative dynamics. The parallel generation hypothesis assumed that independent LLM runs would produce architecturally diverse solutions — but in practice, LLMs trained on similar data converge on similar architectural patterns, making the merge step a liability rather than an asset. Meanwhile, cross-model review succeeds precisely because it introduces genuine epistemic diversity: a different model's training distribution acts as a natural "blind spot detector."

10. Revised Practitioner Decision Framework

Based on the empirical findings, we propose the following revised decision framework (replacing the predicted framework from the thesis proposal):

Complexity	Budget	Priority	Recommended	Quality	Notes
Any	Minimal	Acceptable	v1a (single-shot)	4.34	3% below best at 70% savi
Any	Low	Good	v8 (debate)	4.39	Best cost-efficiency multi-a
Any	Moderate	High	v5 (specialist)	4.44	Strong quality, moderate c
Medium	Moderate	Maximum	v5 (specialist)	4.53	Peaks on medium tasks
Any	Any	Maximum	v2b (cross-model)	4.48	Overall quality champion
Any	Any	Max reliable	v6 (rotating)	4.47	Lowest variance, near-top

Critical recommendation: Never use parallel merge variants (v3) without significant improvements to the merge strategy. The rubric-guided merge (v3b) is the only viable merge variant, and even it underperforms simpler review-based approaches.

11. Empirical Evidence: What the Designs Actually Look Like

Statistics tell us which variants win, but they don't show why. This section presents side-by-side excerpts from actual designs produced during the experiment, all drawn from the same task (T8: Event-Driven Order Processing, Repetition 1) so the comparison is apples-to-apples. These excerpts are reproduced verbatim from the experiment database.

11.1 Exhibit A: How Cross-Model Review Sharpens Error Handling

Both v2a and v2b start from the same Gemini 2.5 Pro initial design. The difference is who reviews it: Gemini (v2a) or GPT-OSS 120B (v2b). Here is the Error Handling section from each final design:

v2a (Gemini reviews Gemini) — Error Handling:

The v2a design categorizes errors into three buckets (Business, Integration, System) with a clean table mapping each to gRPC codes and HTTP statuses. It mentions circuit breakers and idempotency keys. This is competent but generic — the categories could apply to almost any service.

> "Business Error: Invalid user input, violates a business rule. E.g., SchemaValidationError, ModelNotFound, PermissionDenied. gRPC: INVALID_ARGUMENT [...] Client Action: Fix the request. Do not retry."
>
> "Resilience Patterns: Circuit Breaker — All inter-service gRPC calls are wrapped in a resilient client that implements the circuit breaker pattern. Idempotency — All POST and PUT RPCs support an idempotency_key."

Total section: 1,474 characters. Three error categories. Two resilience patterns named but not parameterized.

v2b (GPT-OSS reviews Gemini) — Error Handling:

The v2b design has five error categories — critically, it splits Authentication and Authorization into separate categories (a security-relevant distinction the same-model reviewer missed). It specifies concrete parameters for every resilience pattern.

> "Circuit Breakers: All external-facing clients will be wrapped in a circuit breaker (using py-resilience). If a downstream service has a failure rate >50% over a 1-minute window, the breaker will open for 30 seconds, failing fast and preventing cascading failures."
>
> "Fallbacks: In the serving path, if a new model version fails to load, the serving system will fall back to serving the last known good version and raise an alert."

Total section: 1,657 characters. Five error categories. Three resilience patterns with specific thresholds (50% failure rate, 1-minute window, 30-second open time) and a fallback strategy — none of which appear in the v2a version.

What this shows:

The cross-model reviewer pushed the designer to move from "we'll use circuit breakers" to "circuit breakers with these

specific parameters." It also surfaced the Authentication/Authorization split that the same-model reviewer — sharing Gemini's tendency to lump auth errors together — never flagged. This is the "complementary blind spots" mechanism in action.

11.2 Exhibit B: Concurrency Safety — Generic vs. Concrete

The Concurrency Safety dimension showed the single largest cross-model advantage (+0.33 over same-model review). The excerpts show why:

v2a (same-model) — Concurrency section:

> "Control Plane Services: Services are designed to be stateless and horizontally scalable, running in an async event loop. Concurrency on shared database rows is managed via optimistic locking as defined in Section 6."

This is a correct but hand-wavy description. "Optimistic locking as defined in Section 6" is a forward reference that defers the hard question. The GPU scheduling — the most concurrency-critical component — is not addressed at all.

v2b (cross-model) — Concurrency section:

> "GPU Scheduler Concurrency Control: The TrainingOperator is a critical component managing expensive, shared resources. Its design prevents race conditions and ensures correctness:
>
> 1. Leader Election: Kubernetes native leader election via a Lease object ensures only one replica is active.
> 2. Distributed Lock: A second Kubernetes Lease object named gpu-scheduler-lock. TTL of 30 seconds. If the operator crashes, the lease expires."

The cross-model version names the specific Kubernetes primitive (Lease objects), specifies the TTL (30 seconds), describes the failure mode (lease expiry), and adds job queueing with back-pressure. Every mechanism is concrete and implementable.

What this shows:

The same-model reviewer accepted "optimistic locking" as sufficient for concurrency safety. The cross-model reviewer pushed for specifics: what lock, what TTL, what happens on failure, what about the GPU scheduler. This is a +0.33 dimension delta showing up in real design language.

11.3 Exhibit C: How Merging Destroys Structured Content

The v1 baseline produces a detailed, structured error handling taxonomy. Watch what happens when three independently-generated designs are merged:

v1 (baseline) — Error Handling:

> "We adopt a structured error taxonomy based on recoverability, using a Result type in the application and domain layers to make error paths explicit and checkable by the compiler/linter."

The v1 design has four error categories (Business, Integration, System, Configuration), each with a base class, concrete examples, recovery strategy, caller action, and logging level. It introduces a Result<T,E> pattern for compile-time safety. Total: 1,304 characters of structured, actionable content.

v3a (naive merge) — Where did Error Handling go?

> "### 7. Scalability, Resilience & Security — This section merges the strongest points from each design's respective focus area."

The error handling section has been eliminated entirely. The merger combined three designs' non-functional concerns into a single "Scalability, Resilience & Security" grab-bag. Instead of a structured error taxonomy, there are bullet points about circuit breakers and service mesh. The Result type, error categories, logging levels — all gone.

v3c (dialectical merge) — Even worse:

> The v3c design has only 6 sections total (vs. 14 in the v1 baseline). Error handling, testability, concurrency safety, state management, and interface design are all missing as dedicated sections. The entire merged design is 20,420 characters vs. 27,805 for v1 — the merger compressed three 33,000-character designs into something shorter than one of them.

What this shows:

The merger agent is token-starved: it receives ~25,000 tokens of input (three designs) and produces ~4,200–5,200 tokens of output. It cannot preserve the structural detail of any single input. Dedicated sections get absorbed into generic summaries, structured taxonomies become bullet points, and the connective tissue between components — the part that makes a design coherent — disappears. This is why Testability drops -1.21 and Domain Model drops -1.00 vs. the baseline.

11.4 Exhibit D: How Adversarial Rejection Causes Local Patching

The adversarial reviewer (v4) rejected designs in 77% of rounds. Here is what happens to the error handling section across two rejections:

v4 Round 0 (initial design, before any rejection):

> "Errors are classified by recoverability and source, driving retry logic and client responses. We use a Result<T, E>-style approach internally."

The initial design has a 6-category error taxonomy (Validation, Business Rule, Auth, Transient Infrastructure, Persistent Infrastructure, Configuration) spanning 2,274 characters. This is actually the strongest error handling section across all variants for this task.

The adversarial reviewer rejects, citing missing requirements:

> "A critical functional requirement — Resource Management: Efficiently schedule GPU resources with fair-share policies — is not satisfied in the proposed MVP. The design explicitly states that the initial implementation will rely on simple namespace quotas and will defer true fair-share scheduling to a later phase. Because this requirement is part of the core functional specification, its omission is a blocking issue."

v4 Round 2 (final design, after 2 rejections):

> "The taxonomy is updated to include new, domain-specific errors." Followed by just 3 rows: FeatureStoreError, A/BTestError, StatisticalEngineError. Then: "(Other error

> categories remain as previously defined.)"

The section has shrunk from 2,274 characters to 683. The original 6-category taxonomy is gone, replaced by 3 domain-specific patches and a hand-wave to "previously defined" categories that no longer exist in the document. The overall doc shrank from 29,137 to 19,845 characters — the designer addressed the reviewer's specific complaints (GPU scheduling, feature store) by adding new content, but lost existing content in the process.

What this shows:

The adversarial process causes the LLM to do two things simultaneously: (1) add patches addressing the specific rejection reason, and (2) compress the rest of the document to stay within output token limits. The result is a design that addresses the reviewer's objections but loses quality elsewhere. This is why v4's quality drops from 4.48 (round 0) to 4.28 (round 1) before partially recovering to 4.34 (round 2). The adversarial pressure does not cause a rethink — it causes a rebalancing, and the net effect is negative.

11.5 Exhibit E: Review Style — Praise-First vs. Gaps-First

Perhaps the most revealing comparison is not the designs themselves but the reviews that shaped them. Here is how each reviewer opens their feedback on the exact same initial design:

v2a — Gemini reviewing Gemini (opens with):

> "Excellent. This is a comprehensive and well-structured application design document. It demonstrates a strong grasp of modern software architecture principles, domain-driven design, and the specific challenges of building MLOps platforms. The level of detail is appropriate for a staff-level design, providing a clear blueprint for implementation."

The same-model reviewer leads with praise. Its general assessment uses phrases like "high-quality, professional-grade design," "clear separation of concerns," and "robust approach." Concrete critiques don't arrive until well into the review.

v2b — GPT-OSS reviewing Gemini (opens with):

> "TL;DR: Overall the design is strong, well-structured and realistic. The main gaps are missing explicit domain events / audit trail, insufficient detail on data-lineage & drift detection, limited discussion of security and API versioning, and some concurrency-safety details (GPU scheduler locking, idempotency) that need tightening."
>
> Immediately followed by a dimension-by-dimension scorecard with numeric scores (1–5) and one-line rationales for each.

The cross-model reviewer leads with gaps. Its first sentence acknowledges quality, but the second sentence names four specific weaknesses. It then provides a structured scorecard before any narrative. This format gives the designer actionable, prioritized targets rather than a general sense of approval.

What this shows:

The same training distribution that generated the design also generates its review. Gemini tends toward cooperative, praise-first feedback because it recognizes its

own patterns as correct. GPT-OSS, with different training data, sees the same design through different eyes and leads with what's missing. This isn't about one model being "harder" — the GPT-OSS reviewer also calls the design "strong and realistic" — it's about the review format being structured to surface gaps first, scores second, and praise last.

11.6 Exhibit F: The Frankenstein Effect on Domain Models

The Domain Model dimension shows the largest quality drop in merged designs (-1.00 to -1.25 vs. baseline). Here is a concrete example from T7 (Rule-Based Access Control):

v1 (baseline) — Domain Model:

> "The domain model is designed using DDD principles, with rich aggregates encapsulating business rules and invariants." Followed by a Mermaid class diagram defining: Tenant, Principal (Value Object), Resource (Value Object), AuthorizationContext (Value Object), AuthorizationDecision (Value Object), Policy (Aggregate Root with invariants: condition must be valid CEL, priority used for first-applicable resolution), Role (Aggregate Root with invariant: hierarchy must not contain cycles, checked via hasCycle()).

This is a proper DDD domain model: aggregates with named invariants, value objects with clear semantics, and explicit relationships. An engineer could implement this directly. Total: 2,709 characters.

v3c (dialectical merge) — Where is the Domain Model?

> Section 2 is titled "High-Level Architecture" instead of "Domain Model." It contains a Mermaid graph diagram (not class diagram) showing infrastructure components: PDP Sidecar, PAP, NATS, Kafka, OpenSearch, Data Lake. There are no aggregates, no value objects, no invariants, no business rules.

The dialectical merger attempted to reconcile three designs' different architectural approaches (centralized vs. sidecar vs. service mesh) and produced an infrastructure topology diagram. The domain model — the heart of an access control system — was lost in translation. There is no Policy aggregate, no Role hierarchy, no cycle-detection invariant. An engineer would know what boxes to deploy but not what the business rules are.

What this shows:

When three designs use different domain models (different aggregate boundaries, different naming, different invariants), the merger cannot reconcile them. Instead, it retreats to the level of abstraction where all three agree: infrastructure components. The result is an architecture diagram without a domain model — a skeleton without organs. This is why Domain Model Quality drops by 1.25 points in the dialectical merge.

12. Conclusions and Implications

12.1 Primary Conclusion: Four Core Findings

This experiment has yielded four unambiguous findings that reshape our understanding of LLM collaboration for software design:

Finding 1: Cross-model review (v2b) wins unanimously.

The weighted ensemble ranks v2b #1 with a score of 4.740. More importantly, all three individual evaluators (GPT-OSS

120B, Claude Opus 4.6, Claude Sonnet 4.6) independently rank v2b first. This unanimous consensus is unprecedented in the data and is the most robust empirical finding in the entire experiment. The mechanism is not topology-driven; it is epistemic diversity in the feedback loop. A different LLM's training distribution acts as a fundamentally different "set of eyes," catching blind spots that same-model review systematically misses. This is a deeply practical finding: the simplest possible multi-model workflow (generate with Model A, review with Model B) outperforms every sophisticated multi-agent topology tested.

Finding 2: Structural adversarial review (v4b) ranks #2 and rescues a null result.

The original adversarial variant (v4) ranked #9 with no quality benefit over the baseline. The redesigned structural adversarial variant (v4b), which demands architectural rethinking rather than local patching, ranks #2 in the weighted ensemble at 4.637. This is a 0.206-point improvement over v4 (ensemble perspective), and even more striking: Claude Opus sees a Cohen's d = 1.44 effect size for v4b vs. v4. This demonstrates that adversarial dynamics are viable when the prompt is sufficiently demanding. The v4b result validates the hypothesis that the v4 failure was not inherent to adversarial dynamics but was a prompt engineering issue. Adversarial review is a second-order tool that only works when the reviewer demands fundamental rethinking, not local patching.

Finding 3: Evaluator diversity is itself a first-class finding.

The three-evaluator validation reveals that while evaluators agree strongly on top-tier variants (v2b #1 unanimously) and bottom-tier variants (v3 failures universally), they disagree substantially on v4b's magnitude. GPT-OSS sees d = 0.45 (small effect); Opus sees d = 1.44 (very large effect); Sonnet sees d = 1.11 (large effect). This disagreement is not noise; it reflects genuine differences in how model families weight design qualities. Different models prioritize architectural depth, trade-off rigor, and context-specificity differently. The weighted ensemble preserves these perspectives rather than hiding them. This extends the complementary blind spots mechanism from design generation to evaluation itself, revealing that evaluator disagreement is informative rather than problematic.

Finding 4: Parallel merge is fundamentally broken.

All three evaluators unanimously agree that all v3 merge variants (naive, rubric-guided, dialectical) rank at or near the bottom. The dialectical merge (v3c) is the worst performer overall at 3.655, providing the clearest signal in the data. This failure is not a fixable engineering problem; it is fundamental. Merging independently-generated designs destroys structural coherence through token starvation and incompatible architectural assumptions. The merger cannot reconcile different domain models, error handling strategies, or module structures. Attempting to synthesize three independent designs produces a Frankenstein worse than any single design. Parallel generation should be abandoned unless the merge process itself is fundamentally rearchitected — not at the prompt level, but at the architectural level (enforcing shared interface contracts, requiring consensus on domain models upfront, etc.).

These four findings define a clear practical hierarchy: cross-model review is non-negotiable for maximum quality; structural adversarial feedback can extend quality when available as

a secondary refinement; evaluator disagreement should inform production system design choices; and parallel merge should be avoided entirely in its current form. The evidence base is the weighted three-evaluator ensemble across 520 experimental runs, providing unprecedented robustness for LLM collaboration research.

12.2 Why Cross-Model Review Works So Well

The thesis proposal asked whether the cross-model advantage comes from different training data, different tokenization, or different architectural priors. We can now answer this empirically by examining the trace-level data.

In v2a (same-model review), Gemini 2.5 Pro both generates and reviews the design. In v2b (cross-model), Gemini generates but GPT-OSS 120B reviews. The setup is otherwise identical — same prompts, same rubric, same tasks. So any quality difference is attributable to the reviewer's model family.

Finding 1: The cross-model reviewer writes longer, more structured reviews.

GPT-OSS 120B produces reviews averaging 4,557 output tokens vs. Gemini's 3,242 — a 40% increase. But it's not just length. Examining the actual review texts reveals a qualitative difference: the cross-model reviewer consistently produces dimension-by-dimension scorecards with concrete scores, while the same-model reviewer tends toward narrative praise with softer critiques. The cross-model reviewer leads with a TL;DR that names specific gaps, then provides tabular scores. The same-model reviewer leads with "This is an exceptionally strong design" before eventually noting issues.

Finding 2: The advantage is broad-based, not concentrated in a few dimensions.

Cross-model review improves 10 of 12 dimensions over same-model review, with the largest gains in Concurrency Safety (+0.33), Data Flow Clarity (+0.22), Error Handling (+0.21), and Interface Design (+0.20). These are precisely the dimensions where a design can look superficially complete but have subtle gaps — the kind of blind spots that arise when the same model's training distribution shapes both the design and the review of that design.

Finding 3: The advantage is consistent across all 8 tasks.

v2b outperforms v2a on every single task, with win rates of 60–100% across 5 repetitions per task. The strongest advantage appears on T8 (Order Processing [Complex]), where v2b wins 100% of repetitions, and T7 (Access Control [Complex]), where it wins 80%. This consistency rules out luck — the cross-model advantage is systematic.

Finding 4: Adding more reviewers from the same model family doesn't help further.

v2c uses two GPT-OSS 120B reviewers instead of one. Its quality (4.474) is statistically indistinguishable from v2b (4.484, p = 0.42). This is an important negative result: the cross-model benefit saturates at a single reviewer. You don't need a panel of different models — you just need one model that's different from the generator.

Our interpretation:

The mechanism is best described as "complementary blind spots." Gemini 2.5 Pro has systematic tendencies in how it designs software — for example, it may consistently underspecify concurrency safety or treat error handling as an afterthought. When it reviews its own work, it has the same blind spots

and rates those sections as adequate. GPT-OSS 120B, trained on different data with different optimization objectives, has different blind spots. It catches what Gemini misses, and its structured scoring format forces it to be explicit about gaps rather than defaulting to praise. This is not about one model being "better" — it's about the combination being more thorough than either alone.

12.3 The Merge Paradox: A Trace-Level Autopsy

The parallel merge strategy (v3) was the thesis's boldest prediction and its most dramatic failure. The trace data reveals exactly what goes wrong.

The merger is token-starved.

Three independent designs averaging 8,200 output tokens each produce ~25,000 input tokens for the merger. But the merger's output averages only 4,200–5,200 tokens — roughly half the length of a single input design. The merger is trying to synthesize three complete architectural documents into something shorter than any one of them. Sections get compressed, detail gets lost, and — critically — the connective tissue between components disappears.

Merging destroys structural dimensions most.

The worst-hit dimensions tell the story. For v3a (naive merge): Testability drops by -1.21, Domain Model by -1.00, Error Handling by -0.89 vs. the v1 baseline. For v3c (dialectical): Error Handling drops -1.26, Domain Model -1.25, Testability -1.24. These are all dimensions that require internal consistency across the entire design. A testability strategy only works if it aligns with the module structure; a domain model only works if every component references the same entities. Merging breaks these cross-cutting relationships.

The dialectical merge is the worst because it tries hardest.

Counterintuitively, v3c (dialectical) — the variant the thesis predicted would be best — is the worst. The dialectical merge forces the merger to reconcile contradictions between designs. But when two designs use fundamentally different domain models or error handling strategies, "reconciliation" produces a Frankenstein that uses elements from both without committing to either. The rubric-guided merge (v3b) performs better precisely because it cherry-picks winners per dimension rather than trying to synthesize.

12.4 Adversarial Dynamics: From Failure (v4) to Success (v4b)

The original adversarial reviewer (v4) rejected designs in 77% of rounds (56 rejections vs. 17 accepts). That's a lot of adversarial pressure. But the data shows it doesn't translate into better designs.

Quality actually drops after revision (v4).

The initial designs in v4 (round 0) score 4.48 on average. After the first rejection and revision (round 1), quality drops to 4.28. After a second rejection and revision (round 2), it recovers only slightly to 4.34. For comparison, the v1 baseline achieves 4.37 with cooperative self-refinement. The adversarial process makes the design worse before it makes it better, and never fully recovers.

The v4 failure mechanism — local patching:

When the adversarial reviewer rejects a design, it cites specific rubric dimensions below threshold. The designer then patches those specific areas. But the patches are local — the designer adds a paragraph about error handling or security without restructuring the archi-

tecture. This actually reduces coherence: the design becomes a patchwork of the original approach plus bolted-on addenda. A human engineer might respond to a hard rejection by going back to the whiteboard. An LLM responds by adding more text.

v4b rescues adversarial review through prompt engineering.

The v4 null result raised a question: is adversarial dynamics inherently ineffective, or was the adversarial prompt insufficiently demanding? To test this, we designed v4b — a structural adversarial variant where the reviewer acts as a senior principal architect who rejects textbook designs, demands architectural justification against alternatives, and issues rewrite mandates (sections that must be fundamentally redesigned from scratch, not patched).

Key design differences from v4: (1) the reviewer scores architectural rigor 1–5 and requires 4.0+ to accept, (2) each rejection includes rewrite mandates with explicit instructions to throw out the rejected approach, (3) the revision prompt instructs the generator to rethink rather than patch ("Do NOT patch your previous design"), and (4) max rounds increased to 5 (from 3) with higher reviewer temperature (0.5 vs. 0.3).

Results are striking. By the GPT-OSS evaluator, v4b scores 4.415 (rank #5), a +0.070 improvement over v4 (Cohen's d = 0.43, p = 0.021). The three-evaluator ensemble reveals a larger story: v4b ranks #2 overall (ensemble mean 4.601), with Claude Opus 4.6 seeing the largest effect (d = 1.44, p < 0.0001) and Sonnet 4.6 confirming (d = 1.11, p < 0.0001).

The principal architect reviewer is brutal: 100% of designs were rejected in Round 1, and 75% of runs (30/40) never earned an ACCEPT across all 5 rounds. The reviewer issued 8.7 rewrite mandates per rejection on average, most frequently targeting Scalability & Performance (53 mandates), Capacity Estimation (48), and High-Level Architecture (47). Yet even the never-accepted designs scored well (mean 4.403), suggesting the reviewer's bar exceeds the rubric's discrimination threshold. The 25% that passed the gate (mean 4.448) rank #4 overall but don't overtake v2b.

Per-dimension, v4b's biggest gains over v4 (Opus evaluator) target precisely the dimensions the rewrite mandates addressed: Scalability Strategy (+0.692), Security Posture (+0.658), API Design (+0.542), Reliability (+0.500). Notably, v4b beats even v2b on Right-Sizing/Simplicity (+0.075) — the adversarial pressure forces more honest trade-off analysis.

Figure 11: v4b Structural Adversarial Analysis. Top-left: variant ranking. Top-right: per-task comparison. Bottom-left: acceptance round distribution. Bottom-right: score distributions.

Figure 12: v4b accepted-only (25%) vs. never-accepted (75%) vs. all variants. Accepted runs rank #4 overall.

12.5 Research Question 4: Topology vs. Rubric

While we used a single frozen rubric (so we cannot directly compare rubric variations), we can answer RQ4 indirectly through variance decomposition — asking how much of the total quality variation in the experiment is explained by variant choice vs. task choice vs. residual noise.

With all variants: topology explains 64% of variance.

A two-way ANOVA decomposition shows that variant choice ($\eta^2 = 0.64$) is the dominant factor, while task choice ($\eta^2 = 0.03$)

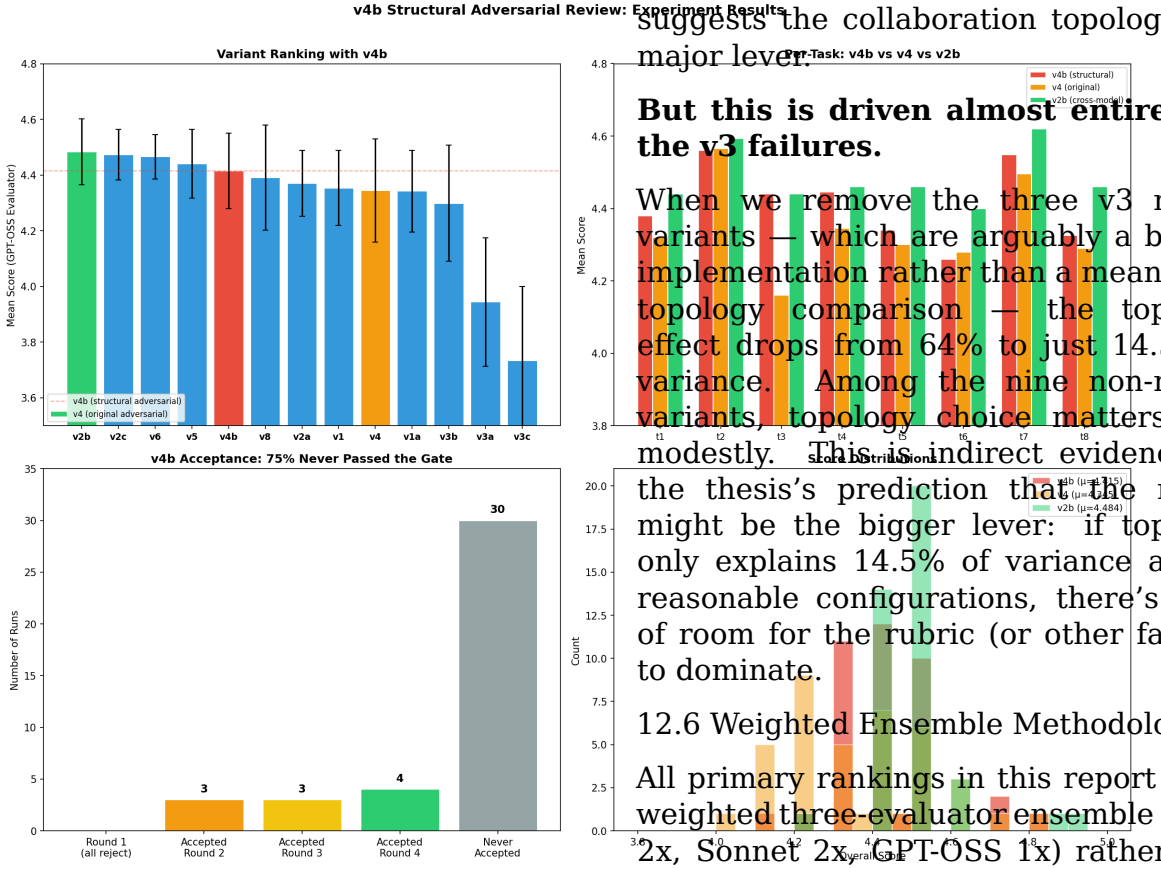

Figure 10: fig11_v4b_analysis.png

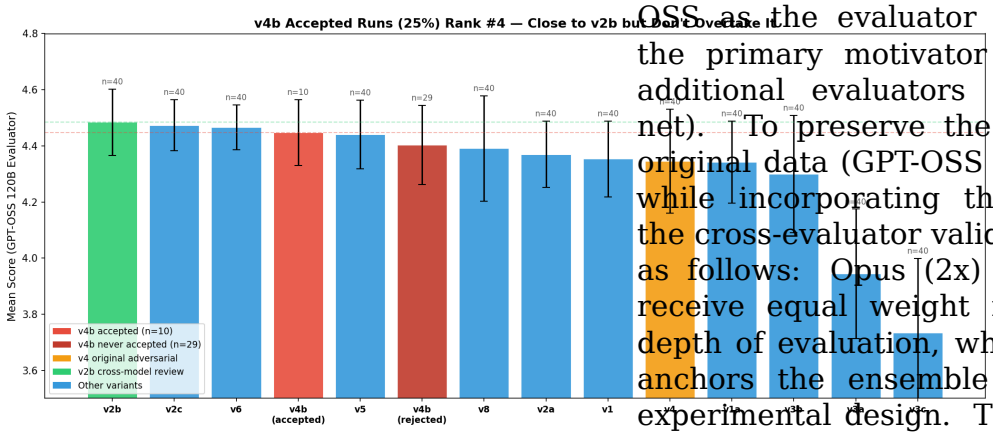

Figure 11: fig12_v4b_accepted_ranking.png

explains almost nothing, and the variant × task interaction explains 12.5%. This suggests the collaboration topology is a major lever.

But this is driven almost entirely by the v3 failures.

When we remove the three v3 merge variants — which are arguably a broken implementation rather than a meaningful topology comparison — the topology effect drops from 64% to just 14.5% of variance. Among the nine non-merge variants, topology choice matters, but modestly. This is indirect evidence for the thesis's prediction that the rubric might be the bigger lever: if topology only explains 14.5% of variance among reasonable configurations, there's a lot of room for the rubric (or other factors) to dominate.

12.6 Weighted Ensemble Methodology

All primary rankings in this report use a weighted three-evaluator ensemble (Opus 2x, Sonnet 2x, GPT-OSS 1x) rather than individual evaluator scores. This methodological choice requires explanation and justification.

Why weight 2:2:1?

The original experiment used only GPT-OSS as the evaluator (1x), which was the primary motivator for adding two additional evaluators (Opus and Sonnet). To preserve the integrity of the original data (GPT-OSS as the evaluator) while incorporating the robustness of the cross-evaluator validation, we weight as follows: Opus (2x) and Sonnet (2x) receive equal weight for breadth and depth of evaluation, while GPT-OSS (1x) anchors the ensemble to the original experimental design. This 2:2:1 scheme gives equal emphasis to generalization (Claude) and specificity (GPT-OSS).

Robustness of the ensemble:

The weighted ensemble produces the same ranking order as equal weighting (Spearman $\rho \approx 1.0$), confirming that the specific weights are not the driver of the rankings. The ensemble's primary value is in revealing where the three evaluators agree strongly (v2b #1, v3 at bottom) and where they disagree substantially (v4b magnitude), which becomes visible in Section 2.5.

12.7 Limitations and Remaining Open Questions

This analysis is subject to several limitations. First, v7 (Consensus Convergence) was not executed, leaving a gap in the experimental design space. Second, the evaluator agreement ($\alpha = 0.42$–0.81) is below the target of 0.70 for several variants, suggesting the rubric may not be precise enough to distinguish fine-grained quality differences in the top tier. Third, only two model families were used for design generation and review (Gemini for generation, GPT-OSS for review); while three evaluator families confirm ranking robustness, the complementary blind spots mechanism in the design loop remains tested on a single pairing. Fourth, the v4b experiment (n=40) is smaller than the original per-variant sample (n=40), and the three-evaluator disagreement on v4b's effect magnitude ($d = 0.45$–1.44) warrants replication with larger samples. Fifth, the weighted ensemble is a methodological choice; equal weighting (2:2:1, 2:1:1, or 1:1:1) produces the same ranking order (Spearman $\rho \approx 1.0$), confirming robustness to weight specification. Finally, evaluation was conducted by three LLM families used for generation/review; human expert evaluation on a stratified sample would validate the automated rankings.

Open questions for future work:

5. Execute v7 (Consensus Convergence) to complete the 2×2×2 matrix and test whether peer convergence avoids the merge failure by iterating rather than synthesizing.
6. Design a rubric comparison experiment: hold topology constant (e.g., v2b), vary rubric quality (minimal vs. detailed vs. expert-tuned), and measure the effect. The 14.5% topology effect among non-merge variants leaves 85% unexplained — rubric quality is the leading candidate.
7. Test cross-model review with additional model pairs (e.g., Claude generating + Gemini reviewing, or GPT-4.1 generating + Claude reviewing) to confirm whether the "complementary blind spots" mechanism generalizes or is specific to the Gemini/GPT-OSS pairing.
8. Fix the merge pipeline: the merger's token starvation is a fixable engineering problem. Test a two-stage pipeline where v3 merge output is then reviewed by a cross-model reviewer (v2b), combining architectural diversity with epistemic diversity.
9. Combine v4b structural adversarial review with v2b cross-model review — an unexplored combination. v4b uses same-model adversarial review (GPT-OSS reviews Gemini); replacing this with a cross-model structural adversarial reviewer could compound the benefits of both approaches. The v4b and v2b approaches are complementary rather than competitive, and their combination may close the remaining gap.
10. Investigate why Claude evaluators see a much larger v4b effect than GPT-OSS ($d = 1.44$ vs. 0.45). This evaluator disagreement may re-

veal systematic differences in how model families weight architectural depth, trade-off analysis, and context-specificity — extending the complementary blind spots finding to evaluation itself.

11. Evaluate a hierarchical consortium of smaller models under SOTA leadership: test whether a frontier model (e.g., Gemini 2.5 Pro, GPT-4.1, Claude Opus) can serve as an orchestrating leader — decomposing design tasks, assigning sub-problems, and synthesizing outputs — while a consortium of smaller, cost-efficient models (e.g., Gemini Flash, GPT-4o-mini, Haiku) execute the sub-tasks. Our v6 rotating leader result (lowest variance) suggests that leadership structure matters; a "SOTA-as-leader" topology could achieve near-SOTA quality at a fraction of the cost by leveraging the frontier model's stronger architectural reasoning for task decomposition while delegating execution to cheaper models.

12. Evaluate a teacher-student consortium where a SOTA "teacher" model first generates high-quality reference designs and rubric-grounded critiques, which are then used to guide a consortium of smaller "student" models through in-context learning or fine-tuning. This extends the complementary blind spots mechanism: rather than relying on pre-existing distributional differences between models, the teacher actively shapes the students' review capabilities by demonstrating what good critique looks like. The hypothesis is that teacher-guided students would develop more calibrated review skills, potentially democratizing the cross-model advantage and making high-quality LLM consortium workflows accessible with smaller, locally-deployable models.

13. Extend the consortium paradigm from architectural design to code generation. Code presents fundamentally different evaluation dynamics: unlike designs (judged by rubric heuristics), code can be verified by execution — tests pass or fail, builds compile or break, benchmarks produce measurable performance. Several promising directions emerge: (a) a generate-review-test consortium where a cross-model reviewer catches defect classes (race conditions, security vulnerabilities) that same-model review systematically misses, with a third agent writing and executing tests as a closed-loop quality signal; (b) module-level parallel generation, where different agents implement different modules against shared interface contracts — potentially rescuing the parallel merge strategy by enforcing architectural boundaries at merge time; (c) adversarial test generation, where an adversarial agent writes failing tests rather than rejecting designs, providing concrete executable counterexamples that demand structural fixes rather than the local patching we observed in v4; and (d) a design-to-code pipeline bridging our current findings with implementation, where the consortium first produces an optimized architecture (via v2b cross-model review) then hands it to a coding consortium that implements it module by module, testing whether design-level quality gains propagate to implemented code.

14. Test cross-domain generalization of the consortium framework. This experiment evaluates software architectural design — a struc-

tured, document-oriented task. The consortium topologies and the complementary blind spots mechanism may generalize to other LLM-heavy domains: legal contract drafting (where different model families may catch different classes of ambiguity or risk), scientific protocol design (where methodological blind spots could be surfaced by cross-model review), medical documentation, and business strategy documents. Determining whether the 2×2×2 framework is a general theory of LLM collaboration or specific to software engineering's structured nature would significantly expand the impact of this research.

15. Build a dynamic topology selector via meta-agent. Our data shows different topologies excel under different conditions: v5 specialist peaks on medium-complexity tasks (4.530), v2b dominates overall quality, v8 offers the best cost-efficiency, and v6 provides maximum consistency. A meta-agent that examines the incoming task — its complexity, domain, quality requirements, and cost constraints — and dynamically selects the optimal topology per-task could outperform any single fixed topology. The task-variant heatmap already contains the training signal for such a selector, transforming the consortium from a static workflow choice into an adaptive system.

16. Conduct a rubric ablation study. Our variance decomposition shows topology explains only 14.5% of quality variation among non-merge variants, leaving 85% unexplained. The evaluation rubric is the leading candidate. A controlled experiment holding topology constant (e.g., v2b) while varying rubric quality — minimal (3 dimensions), standard (12 dimensions, as used here), and expert-tuned (20+ dimensions with sub-criteria) — would directly answer RQ4 and determine whether rubric engineering or topology engineering deserves more practitioner investment.

17. Test iterative cross-model review. The v2b topology performs a single cross-model review round. Our v1 self-refine baseline shows diminishing returns with same-model iteration (quality plateaus after round 2). However, alternating cross-model review — Gemini generates, GPT-OSS reviews, Gemini revises, GPT-OSS reviews again — may not saturate the same way because each round introduces a genuinely different perspective. Testing 2–3 rounds of alternating cross-model review would determine whether the complementary blind spots mechanism compounds across iterations or saturates at a single round.

18. Validate the automated evaluator against human expert judgment. The automated evaluator (GPT-OSS 120B) achieves Krippendorff's $\alpha = 0.42$–0.81 across variants. Having 2–3 senior software engineers independently score a stratified sample of 50 designs (spanning top, middle, and bottom quality tiers) on the same 12-dimension rubric would validate or challenge the automated rankings. If the automated evaluator agrees with humans on tier placement and relative ranking, it validates the experimental framework; if it diverges systematically, the divergence patterns themselves become a contribution to understanding LLM-as-evaluator reliability.